

A Tensorflow/Keras Extension for ML with Imbalanced Data

Tommy Galletta, Daniel Acosta, Philip Chan, Ming Zhang

Florida Institute of Technology

Introduction

- Forecasting infrequent/rare heliophysics events (e.g. SEP events) is challenging
- Issue is called imbalanced (long-tailed/skewed) data distributions in Machine Learning (ML)
- Tensorflow/Keras extension for ML
 - imbalanced data distributions
 - classification (e.g. SEP vs. non-SEP) and regression (e.g. SEP intensity) tasks
 - tabular and image data

Overview of the tool

- Training methods
 - Evaluation metrics
 - Visualization of the latent/representation space
 - Explanation of predictions
-
- Syntax and terminology follow Keras/Tensorflow

Overview of the tool

- Training methods
- Evaluation metrics
- Visualization of the latent/representation space
- Explanation of predictions

Training Methods

- 3 fit methods [Keras uses the term “fit” for “train”]
 - Regular fit
 - Balanced fit
 - Decoupled fit
- Additional options
 - Decoder generation
 - Validation split

Training Methods

- 3 fit methods [Keras uses the term “fit” for “train”]
 - Regular fit
 - Balanced fit
 - Decoupled fit
- Additional options
 - Decoder generation
 - Validation split

Regular Fit

- Syntax nearly identical to default Keras syntax
- Automatically performed batch stratification
 - Stratification achieves a similar data distribution in all batches
 - i.e. rare samples are present in each batch

```
# Assuming data has already been loaded, and
# model structure has been previously defined

model = imbal.classification.Model(
    inputs=input_layer,
    outputs=output_layer
)

model.compile(
    optimizer=optimizers.Adam(learning_rate=2e-4),
    loss='binary_crossentropy',
)

model.fit(
    x_train,
    y_train,
    epochs=200,
    batch_size=64
)
```

Balanced Fit (Classification)

- Weight each class equally, rather than each sample equally
- Custom distribution is an option

	Cats	Dragons
Original distribution	99	1
Balanced distribution	50	50
Sample weight	$50/99$	$50/1 = 50$
Custom distribution	60	40
Sample weight	$60/99$	$40/1 = 40$

Balanced Fit (Classification)

- Minimal changes compared to regular fit
- Automatically performs a class-balanced sampling weighting
 - Each class is weighted as being equally “important”

```
0, 0, 0, 0, 0, 1 --> class labels
1, 1, 1, 1, 1, 1 = 6 (default weights)
0.6, 0.6, 0.6, 0.6, 0.6, 3 = 6 (class-balanced)
```

```
# Assuming data has already been loaded, and
# model structure has been previously defined

model = imbal.classification.Model(
    inputs=input_layer,
    outputs=output_layer
)

model.compile(
    optimizer=optimizers.Adam(learning_rate=2e-4),
    loss='binary_crossentropy',
)

model.balanced_fit(
    x_train,
    y_train,
    epochs=200,
    batch_size=64
)
```

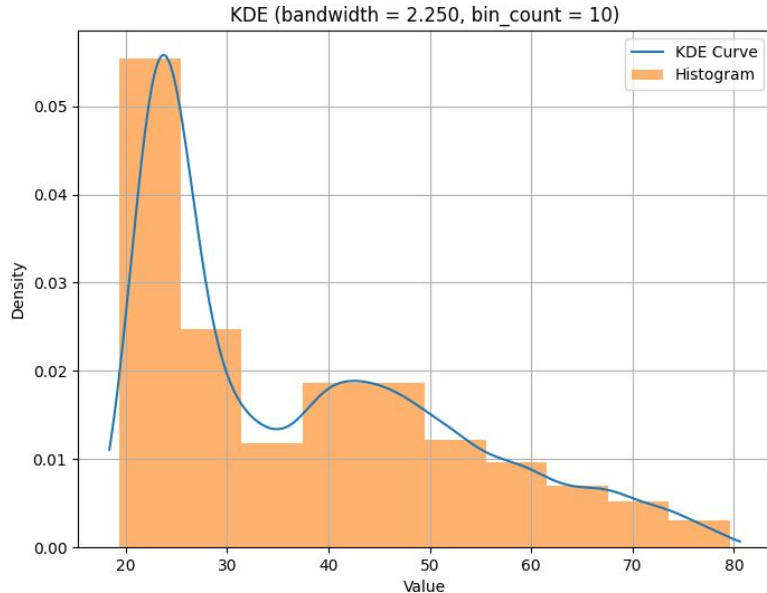
Balanced Fit (Regression)

1. Find an appropriate bandwidth for kernel density estimation (KDE)

2. Find probability density (d) for each sample via KDE

3. Pass densities to `balanced_fit`

- Convert densities to sample weights ($1/d$)
 - Low density $\rightarrow$ High weight
 - High density $\rightarrow$ Low weight
- Optionally, pass custom sample weights



Balanced Fit (Regression)

- Minimal changes compared to regular fit
- Sample densities (or custom sample weights) must be provided.
- Sample weight is inversely proportional to its density

```
0, 0, 0, 0.2, 0.4, 1, 5 --> labels
0.9, 0.9, 0.9, 0.9, 0.6, 0.2, 0.2 --> densities
1.1, 1.1, 1.1, 1.1, 1.7, 4.1, 4.4 --> 1 / densities
0.5, 0.5, 0.5, 0.5, 0.8, 2, 2.2 --> balanced weights
```

```
# Assuming data has already been loaded, and
# model structure has been previously defined

model = imbal.regression.Model(
    inputs=input_layer,
    outputs=output_layer
)

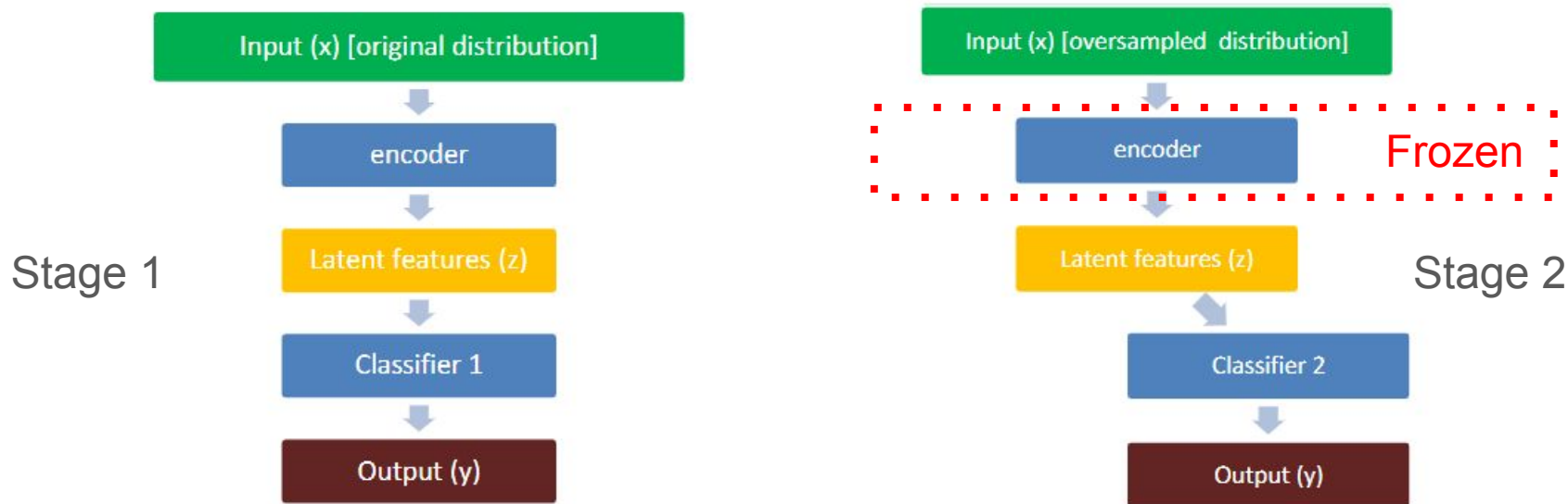
model.compile(
    optimizer=optimizers.Adam(learning_rate=2e-4),
    loss='binary_crossentropy'
)

1 bandwidth = imbal.regression.fit_kde(y_train)
2 densities = imbal.regression.get_sample_densities(
    y_train,
    bandwidth
)

3 model.balanced_fit(
    x_train,
    y_train,
    sample_density=densities,
    epochs=200,
    batch_size=64
)
```

Decoupled Fit (representation vs classifier training)

- cRT (classifier ReTraining) (Kang et al., 2020) [1,976 citations]
- 2 stages of training
 1. Learn representations in latent space using the original distribution
 2. Freeze the encoder and re-train the classifier using a balanced distribution



Decoupled Fit

- Classifier re-training (cRT)
 - Decoupled fit for regression is available as `rRT_fit` under `imbal.regression.Model`

```
# Assuming data has already been loaded, and
# model structure has been previously defined

model = imbal.classification.Model(
    inputs=input_layer,
    outputs=output_layer
)

model.compile(
    optimizer=optimizers.Adam(learning_rate=2e-4),
    loss='binary_crossentropy',
)

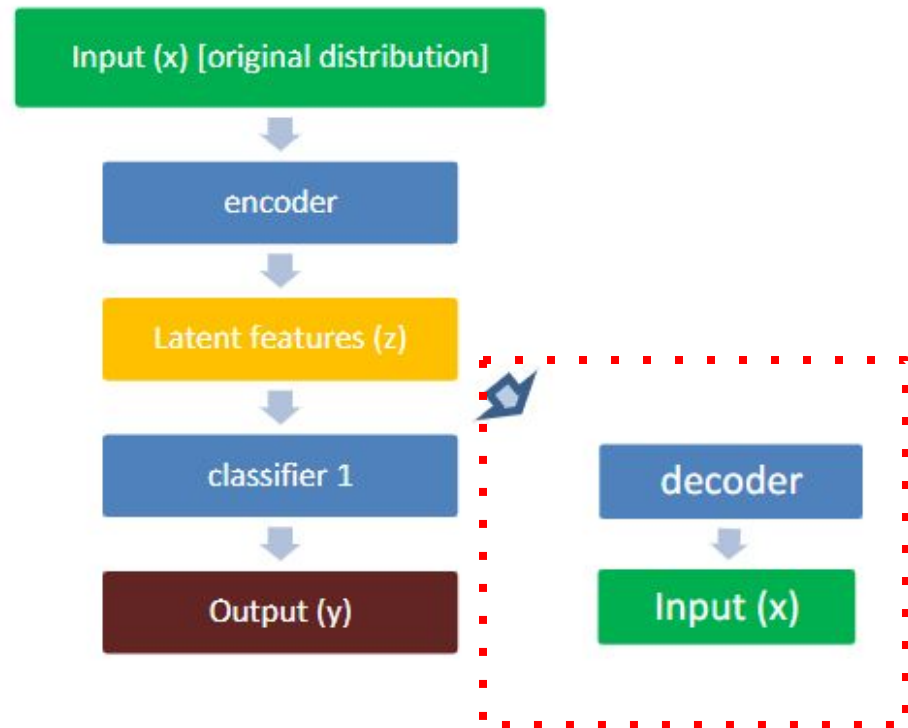
model.cRT_fit(
    x_train,
    y_train,
    epochs=200,
    batch_size=64
)
```

Training Methods

- Three fit methods [Keras uses the word “fit” for training]
 - Regular fit
 - Balanced fit
 - Decoupled fit
- Additional options
 - Decoder generation
 - Validation split

Decoder Generation

- Enhancing representation via autoencoder (encoder + decoder)
- cRT with autoencoder (cRT+AE) in stage 1 (Griessler, 2023)
 - 3rd out of 13 in Probability Forecast at SEPVAL 2023 (Whitman et al., SHINE Workshop 2024)



Decoder Generation

- Compatible with all 3 fit/training methods

```
# Assuming data has already been loaded, and
# model structure has been previously defined

model = imbal.classification.Model(
    inputs=input_layer,
    outputs=output_layer
)

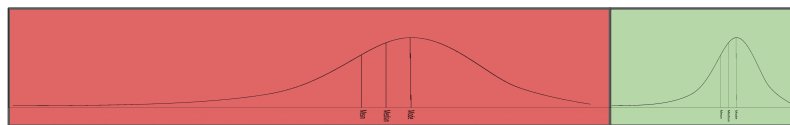
model.compile(
    optimizer=optimizers.Adam(learning_rate=2e-4),
    loss='binary_crossentropy',
    generate_decoder_branch=True
)

model.fit(
    x_train,
    y_train,
    epochs=200,
    batch_size=64
)
```


Validation Split

- (Unseen) validation set
 - Reduce overfitting
 - Determine the best class/sample weights from a list of candidates
- Validation split is stratified

— Original training set —



Training

Validation
(unseen)

Validation Split

- Compatible with all 3 fit/training methods

```
# Assuming data has already been loaded, and
# model structure has been previously defined

model = imbal.classification.Model(
    inputs=input_layer,
    outputs=output_layer
)

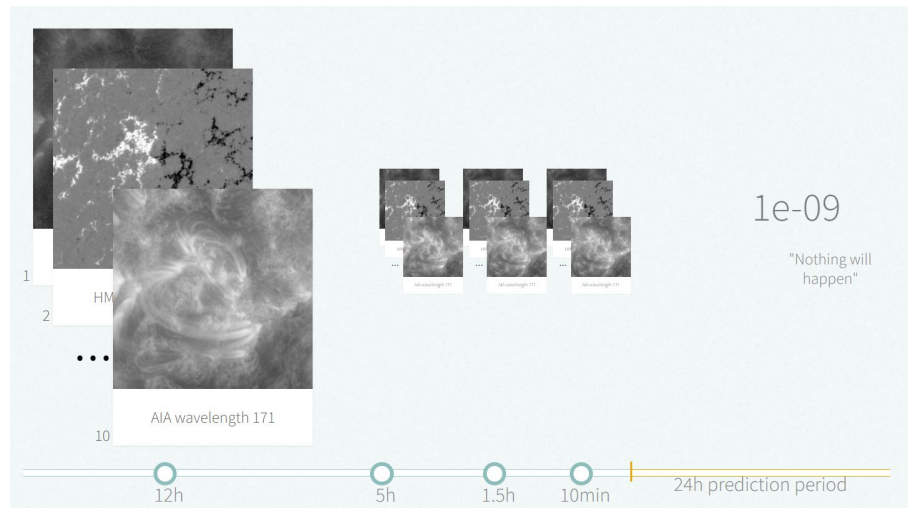
model.compile(
    optimizer=optimizers.Adam(learning_rate=2e-4),
    loss='binary_crossentropy'
)

model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=200,
    batch_size=64
)
```

Tutorials on Training Methods with Heliophysics Data

Tutorial: Image Regression

- Original SDOBenchmark dataset:
 - 10 images at four different timestamps
 - Each image is 256x256
 - Predict the log peak flux within the 24hr prediction period
- Subset for tutorial:
 - One image from the 10min timestamp
 - Each image is 128x128
 - Log peak flux ≥ -4 is an X-class flare
 - Approx. 2% of training data is X-class flares



Tutorial: Image Regression (Regular vs Balanced)

```
model.compile(  
    optimizer=optimizers.Adam(learning_rate=LEARNING_RATE),  
    loss='mse',  
    metrics=['mae']  
)  
  
model.fit(  
    x_train,  
    y_train,  
    epochs=EPOCHS,  
    batch_size=BATCH_SIZE,  
)
```

```
Number of test samples with log10 flux < -4: 586  
Number of test samples with log10 flux >= -4: 14  
19/19 _____ 0s 14ms/step  
MAE for log10 flux < -4: 1.383  
MAE for log10 flux >= -4: 2.944
```

```
model.compile(  
    optimizer=optimizers.Adam(learning_rate=LEARNING_RATE),  
    loss='mse',  
    metrics=['mae']  
)  
  
model.balanced_fit(  
    x_train,  
    y_train,  
    sample_density=sample_densities,  
    epochs=EPOCHS,  
    batch_size=BATCH_SIZE,  
)
```

```
Number of test samples with log10 flux < -4: 586  
Number of test samples with log10 flux >= -4: 14  
19/19 _____ 1s 15ms/step  
MAE for log10 flux < -4: 1.177  
MAE for log10 flux >= -4: 1.619
```

Tutorial: Tabular Classification

- CME and SEP associations
- Original dataset:
 - 1,531 CME events
 - 22 features, with measurements from DONKI and CDAW, and additional derived features
 - Predict if an SEP event ($\ln(\text{peak intensity}) \geq \ln(10)$) is going to occur
- Subset for tutorial:
 - 1,531 CME events
 - 7 features, directly from DONKI and CDAW
 - DONKI_speed_norm
 - DONKI_longitude_norm
 - DONKI_latitude_norm
 - DONKI_half_width_norm
 - CDAW_Linear_Speed_norm
 - CDAW_MPA_norm
 - Halo
 - Approx. 2% of training data is SEP event

Tutorial: Tabular Classification (Regular vs Balanced+AE)

```
model.compile(loss="binary_crossentropy",
              optimizer="adam",
              metrics=[tf.keras.metrics.F1Score(threshold=0.5, name="F1Score"),
                      imbal.metrics.HeidkeSkillScore(threshold=0.5, name="HSS")
              ],
)

model.fit(x_train,
        y_train,
        batch_size=batch_size,
        epochs=max_epochs,
        )
```

Epoch 300/300

48/48 ————— 0s 2ms/step -

24/24 ————— 0s 3ms/step -

Test Loss: 0.0476

Test F1Score: 0.6897

Test HSS: 0.6837

```
model.compile(loss="binary_crossentropy",
              optimizer="adam",
              metrics=[tf.keras.metrics.F1Score(threshold=0.5, name="F1Score"),
                      imbal.metrics.HeidkeSkillScore(threshold=0.5, name="HSS")],
              generate_decoder_branch=True,
              )

class_weights = {0: 0.9, 1: 0.1}

model.balanced_fit(x_train,
                  y_train,
                  class_weight=class_weights,
                  batch_size=batch_size,
                  epochs=max_epochs,
                  )
```

Epoch 300/300

48/48 ————— 0s 3ms/step -

24/24 ————— 0s 3ms/step -

Test Loss: 0.0774

Test F1Score: 0.7339

Test HSS: 0.7277

Overview of the tool

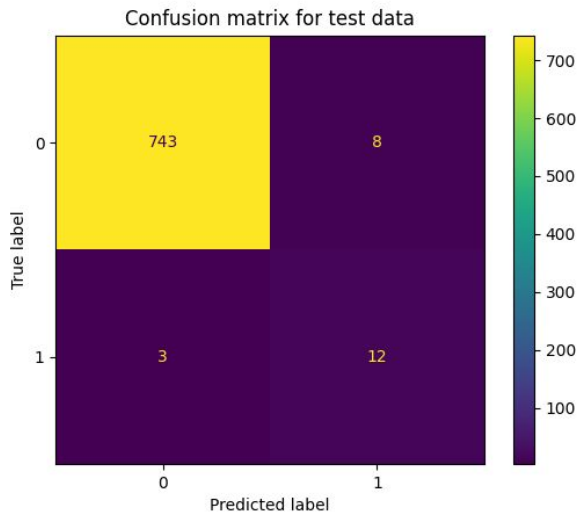
- Training methods
- Evaluation metrics
- Visualization of the latent/representation space
- Explanation of predictions

Evaluation Metrics (Classification) – HSS, TSS, AUROC

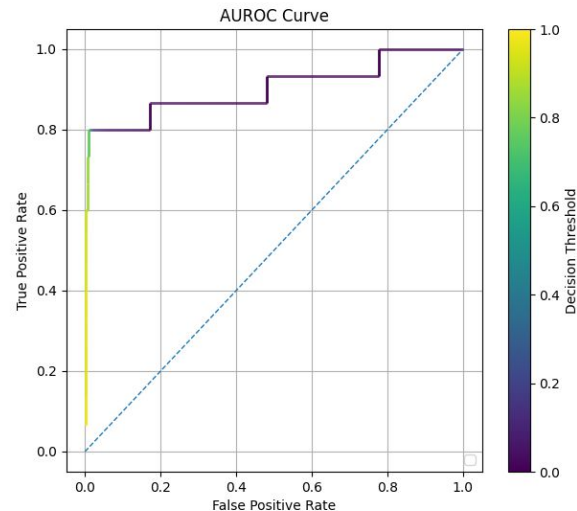
```
model.compile(loss="binary_crossentropy",  
              optimizer="adam",  
              metrics=[imbal.metrics.TrueSkillStatistic(threshold=0.5, name="TSS"),  
                      imbal.metrics.HeidkeSkillScore(threshold=0.5, name="HSS")],  
              )
```

```
auc_metric = keras.metrics.AUC(num_thresholds=50)  
auc_metric.update_state(y_test, y_pred)
```

```
imbal.classification.plot_confusion_matrix(y_test, y_pred)
```



```
imbal.classification.plot_roc(y_test, y_pred)
```



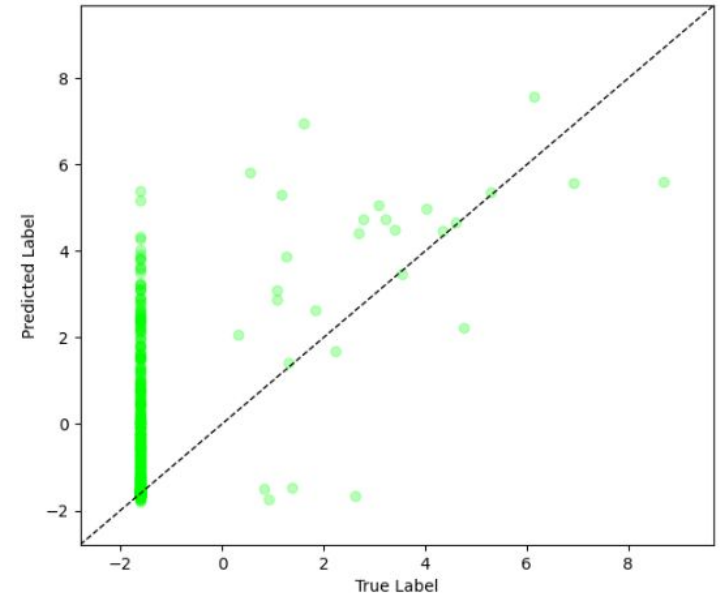
Evaluation Metrics (Regression) – MAE, MSE, PCC

```
model.compile(loss="mean_squared_error",  
              optimizer="adam",  
              metrics=[keras.metrics.MeanAbsoluteError(name="mae")],  
              )
```

```
mse_metric = keras.metrics.MeanSquaredError(name="mse")  
mse_metric.update_state(y_test, y_pred)  
mse = mse_metric.result()
```

```
pcc_metric = keras.metrics.PearsonCorrelation(name="pcc", axis=0)  
pcc_metric.update_state(y_test, y_pred)  
pcc = pcc_metric.result().numpy()
```

```
imbal.regression.plot_true_vs_predictions(y_test, predictions)
```



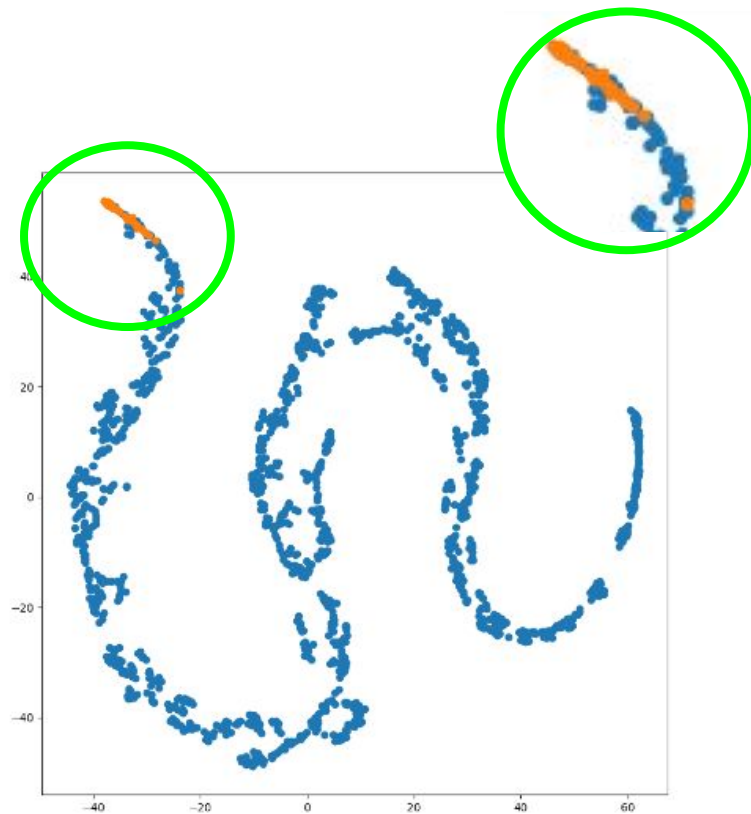
Overview of the tool

- Training methods
- Evaluation metrics
- Visualization of the latent/representation space
- Explanation of predictions

Visualization of the Latent/Representation Space

- t-SNE (Van der Maaten and Hinton, 2008)
[58,173 citations]

```
imbal.classification.tsne_visualization(  
    model,  
    x_train,  
    y_train.reshape(-1),  
    ...  
)
```



Overview of the tool

- Training methods
- Evaluation metrics
- Visualization of the latent/representation space
- Explanation of predictions

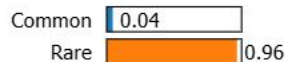
Explanation of Predictions (LIME)

- LIME (Ribeiro et al., 2016)
[33,215 citations]
- Also supports image explanation
 - Highlights important regions of image

```
imbal.classification.lime_explain_tabular_sample  
x_test[target_sample_index],  
model,  
x_train,  
...  
)
```

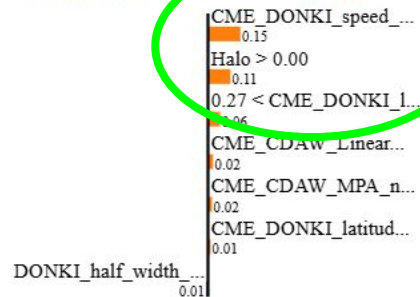
Explanation for "Rare" (Actual label: Rare)

Prediction probabilities



Common

Rare

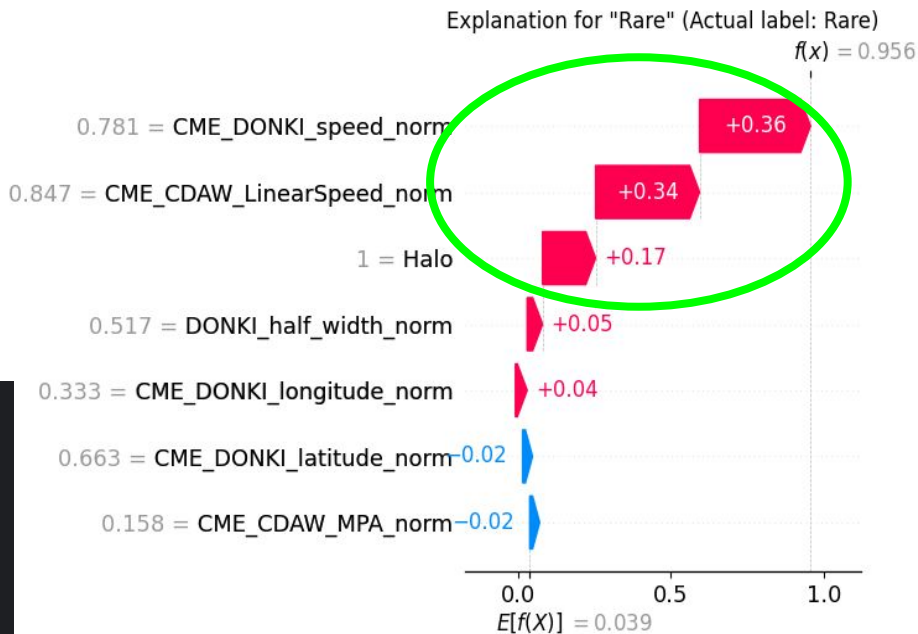


Feature	Value
CME_DONKI_speed_norm	0.78
Halo	1.00
CME_DONKI_longitude_norm	0.33
CME_CDAW_LinearSpeed_norm	0.85
CME_CDAW_MPA_norm	0.16
CME_DONKI_latitude_norm	0.66
DONKI_half_width_norm	0.52

Explanation of Predictions (SHAP)

- SHAP (Lundberg, and Lee, 2017) [56,718 citations]

```
imbal.classification.shap_explain_tabular_sample(  
    x_test[target_sample_index],  
    model,  
    x_train,  
    ...  
)
```



Resources with the tool

- Repository: <https://github.com/fltech-imbal/imbal>
- Documentation: <https://fltech-imbal.github.io/imbal>
 - Documentation of each function
 - Tutorials based on heliophysics tabular/image data
 - Installation guide

Concluding Remarks

- Tensorflow/Keras extension for ML
 - imbalanced data distributions
 - classification and regression tasks
 - tabular and image data
- The extension/tool includes
 - training methods
 - evaluation metrics
 - visualization of latent/representation space
 - explanation of predictions
- Feedback (issues or suggestions) is welcome

Thank you

<https://fltech-imbal.github.io/imbal/>

(with contact info)

Questions?



References

- Griessler (2023). Forecasting >100 MeV SEP Events and Intensity based on CMEs and other Solar Activities using Machine Learning
- Kang, B., Xie, S., Rohrbach, M., Yan, Z., Gordo, A., Feng, J., & Kalantidis, Y. (2020). Decoupling representation and classifier for long-tailed recognition, ICLR.
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *NeurIPS*.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). " Why should i trust you?" Explaining the predictions of any classifier. *SIGKDD* (pp. 1135-1144).
- Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of machine learning research*, 9(11).
- Whitman et. al (2024). First Results from NASA's SEP Model Validation (SEPVAL) Effort. SHINE Workshop.

Related Work

- Imbalanced-learn – <https://imbalanced-learn.org>
 - Undersampling of majority samples
 - Oversampling of minority samples
 - E.g. generating synthetic samples via interpolation
 - Tabular and image data
 - Classification tasks only
 - HSS and TSS are not included.